

Bronze Layer — Data Warehouse Project

1. Overview

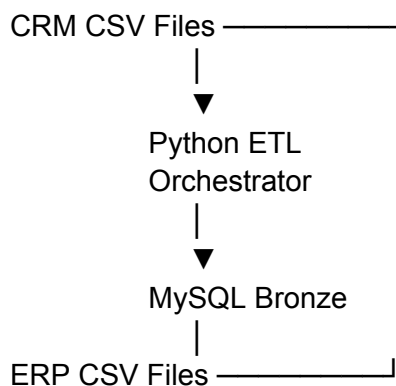
The **Bronze Layer** is the first layer of the data warehouse. Its purpose is to ingest data from source systems into the data warehouse with minimal transformation.

In this project, the source data is provided as CSV files from two source systems:

- **CRM** — Customer Relationship Management data
- **ERP** — Enterprise Resource Planning data

The Bronze layer stores this source data in MySQL tables.

Bronze Layer Flow



The main principle followed is:

Load the source data into Bronze with minimal transformation and preserve the source structure as much as possible.

2. Source Data

The project contains two source systems.

CRM Source

The CRM system contains:

1. Customer information
2. Product information
3. Sales transaction information

Files:

Bronze Layer — Data Warehouse Project

```
source_crm/  
├── cust_info.csv  
├── prd_info.csv  
└── sales_details.csv
```

ERP Source

The ERP system contains:

1. Customer location information
2. Customer demographic information
3. Product category information

Files:

```
source_erp/  
├── loc_a101.csv  
├── cust_az12.csv  
└── px_cat_g1v2.csv
```

3. Bronze Database

A MySQL database was created for the Bronze layer:

datawarehouse_bronze

The Bronze database contains six tables.

Source	Bronze Table	Purpose
CRM	<code>crm_cust_info</code>	Customer information
CRM	<code>crm_prd_info</code>	Product information
CRM	<code>crm_sales_details</code>	Sales transactions
ERP	<code>erp_loc_a101</code>	Customer location
ERP	<code>erp_cust_az12</code>	Customer demographic information
ERP	<code>erp_px_cat_g1v2</code>	Product category information

4. Bronze Table Design

4.1 CRM Customer

```
CREATE TABLE crm_cust_info (  
  cst_id      INT,  
  cst_key     VARCHAR(50),  
  cst_firstname VARCHAR(50),  
  cst_lastname  VARCHAR(50),  
  cst_marital_status VARCHAR(50),  
  cst_gndr     VARCHAR(50),  
  cst_create_date DATE  
);
```

This table stores customer information received from the CRM source.

4.2 CRM Product

```
CREATE TABLE crm_prd_info (  
  prd_id     INT,  
  prd_key    VARCHAR(50),  
  prd_nm     VARCHAR(50),  
  prd_cost   INT,  
  prd_line   VARCHAR(50),  
  prd_start_dt DATETIME,  
  prd_end_dt  DATETIME  
);
```

This table stores product information from CRM.

4.3 CRM Sales

```
CREATE TABLE crm_sales_details (  
  sls_ord_num VARCHAR(50),  
  sls_prd_key VARCHAR(50),  
  sls_cust_id INT,  
  sls_order_dt INT,  
  sls_ship_dt  INT,  
  sls_due_dt   INT,  
  sls_sales    INT,  
  sls_quantity INT,  
  sls_price    INT  
);
```

Bronze Layer — Data Warehouse Project

This table contains sales transaction information.

4.4 ERP Location

```
CREATE TABLE erp_loc_a101 (  
  cid VARCHAR(50),  
  cntry VARCHAR(50)  
);
```

This table contains customer location information.

4.5 ERP Customer

```
CREATE TABLE erp_cust_az12 (  
  cid VARCHAR(50),  
  bdate DATE,  
  gen VARCHAR(50)  
);
```

This table contains additional customer information from ERP.

4.6 ERP Product Category

```
CREATE TABLE erp_px_cat_g1v2 (  
  id VARCHAR(50),  
  cat VARCHAR(50),  
  subcat VARCHAR(50),  
  maintenance VARCHAR(50)  
);
```

This table contains product category and subcategory information.

5. Load Strategy

For the Bronze layer, we implemented a **Full Load** strategy.

The process is:

1. TRUNCATE existing Bronze table

Bronze Layer — Data Warehouse Project

-
- ```
graph TD; A[2. Read complete CSV file] --> B[3. LOAD DATA INFILE]; B --> C[4. Store data in Bronze table]; C --> D[5. Count loaded records]; D --> E[6. Record ETL status];
```
2. Read complete CSV file
  3. LOAD DATA INFILE
  4. Store data in Bronze table
  5. Count loaded records
  6. Record ETL status

For example:

```
TRUNCATE TABLE crm_cust_info;
```

Then:

```
LOAD DATA INFILE
'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/datasets/source_crm/cust_info.csv'
INTO TABLE crm_cust_info
...
```

### Why Full Load?

The current project uses static source CSV files and does not yet have a mechanism for identifying newly inserted or updated records.

Therefore, the Bronze layer currently reloads the complete source dataset each time the pipeline runs.

---

## 6. LOAD DATA INFILE

Instead of inserting CSV records one by one, MySQL's:

```
LOAD DATA INFILE
```

was used for bulk ingestion.

This is much more efficient for loading large CSV files.

The basic process is:

```
graph TD; A[CSV File] --> B[LOAD DATA INFILE];
```

CSV File

LOAD DATA INFILE

## Bronze Layer — Data Warehouse Project



MySQL Bronze Table

We configured:

`FIELDS TERMINATED BY ','`

to indicate that the CSV uses commas as separators.

We also used:

`OPTIONALLY ENCLOSED BY '"'`

to support values enclosed in double quotes.

The header row is skipped using:

`IGNORE 1 LINES`

---

## 7. Handling Empty Values

Some source CSV files contain empty fields.

For example:

`12345,CST001,John,Doe,,M,2024-01-01`

Some columns may contain empty strings.

Instead of directly loading these values, temporary MySQL variables were used.

Example:

`@cst_id`

Then:

`cst_id = NULLIF(@cst_id, "")`

`NULLIF()` converts an empty string into SQL `NULL`.

Therefore:

CSV empty value



`"`



## Bronze Layer — Data Warehouse Project

NULLIF()  
↓  
NULL

This prevents problems when loading empty values into **INT** or **DATE** columns.

This is **basic ingestion handling**, not full business-data cleaning. More detailed cleaning and standardization will be performed in the **Silver layer**.

---

## 8. Python ETL Orchestrator

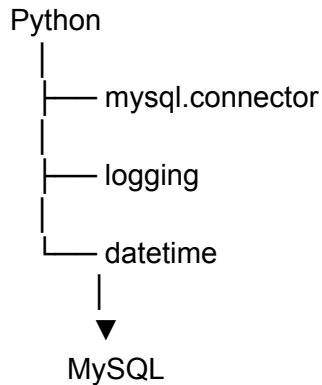
After testing the SQL ingestion manually, we implemented a Python ETL orchestrator.

File:

```
scripts/
├── Bronze/
│ └── BNZ_PYTHON_ORCHETRATOR.py
```

The Python program is responsible for coordinating the Bronze ingestion process.

### Main Technologies



## 9. Why Python Orchestration?

Instead of manually executing six separate SQL loading commands, Python allows us to automate the entire process.

Without orchestration:

Run SQL 1  
Run SQL 2

## Bronze Layer — Data Warehouse Project

Run SQL 3

Run SQL 4

Run SQL 5

Run SQL 6

With Python:

Run Python Script



Load all Bronze tables automatically

The Python script controls:

- Database connection
  - Load order
  - Table truncation
  - CSV ingestion
  - Row counting
  - Logging
  - Audit updates
  - Error handling
  - Connection cleanup
- 

## 10. Database Connection

The Python script uses:

```
import mysql.connector
```

A database configuration is defined:

```
DB_CONFIG = {
 "host": "localhost",
 "user": "root",
 "password": "YOUR_PASSWORD",
 "database": "datawarehouse_bronze"
}
```

A reusable connection function was created:

```
def get_connection():
 return mysql.connector.connect(
 host=DB_CONFIG["host"],
 user=DB_CONFIG["user"],
 password=DB_CONFIG["password"],
```



## Bronze Layer — Data Warehouse Project

```
 database=DB_CONFIG["database"]
)
```

This establishes the connection between Python and MySQL.

---

## 11. Reusable Table Loading Function

Instead of writing the same Python logic six times, a reusable function was created:

```
def load_table(cursor, table_name, load_sql):
```

The function performs:

```
Start audit
 ↓
Log start
 ↓
TRUNCATE table
 ↓
LOAD DATA INFILE
 ↓
Count records
 ↓
Mark SUCCESS
 ↓
Log completion
```

This makes the ETL code reusable and easier to maintain.

---

## 12. ETL Logging

Python logging was implemented using:

```
import logging
```

Example output:

```
2026-10-05 18:18:55 | INFO | Starting Bronze Data Ingestion
2026-10-05 18:18:55 | INFO | Connected to MySQL successfully
2026-10-05 18:18:55 | INFO | Starting FULL load: crm_cust_info
2026-10-05 18:18:56 | INFO | Completed FULL load: crm_cust_info | rows=18494
```

## Bronze Layer — Data Warehouse Project

This provides visibility into what the ETL process is doing while it is running.

---

### 13. ETL Audit Table

A separate audit table was created:

```
CREATE TABLE etl_load_log (
 load_id INT AUTO_INCREMENT PRIMARY KEY,
 table_name VARCHAR(100) NOT NULL,
 load_type VARCHAR(20) NOT NULL,
 start_time DATETIME NOT NULL,
 end_time DATETIME,
 rows_loaded INT DEFAULT 0,
 status VARCHAR(20) NOT NULL,
 error_message TEXT
);
```

This table stores the historical result of each table load.

Example:

| load_id | table_name        | load_type | rows_loaded | status  |
|---------|-------------------|-----------|-------------|---------|
| 1       | crm_cust_info     | FULL      | 18494       | SUCCESS |
| 2       | crm_prd_info      | FULL      | 397         | SUCCESS |
| 3       | crm_sales_details | FULL      | 60398       | SUCCESS |
| 4       | erp_loc_a101      | FULL      | 18484       | SUCCESS |
| 5       | erp_cust_az12     | FULL      | 18484       | SUCCESS |
| 6       | erp_px_cat_g1v2   | FULL      | 37          | SUCCESS |

---

## 14. Difference Between Python Logs and Audit Table

Two types of monitoring were implemented.

### Python Logging

Used for **real-time operational visibility**.

Example:

Starting FULL load: crm\_prd\_info

### MySQL Audit Table

Used for **historical ETL tracking**.

Example:

table\_name: crm\_prd\_info  
rows\_loaded: 397  
status: SUCCESS

Therefore:

Python Logging

↓

"What is happening now?"

MySQL Audit Table

↓

"What happened during previous runs?"

---

## 15. Error Handling

The Python orchestrator contains exception handling:

try:

...

except Exception as e:

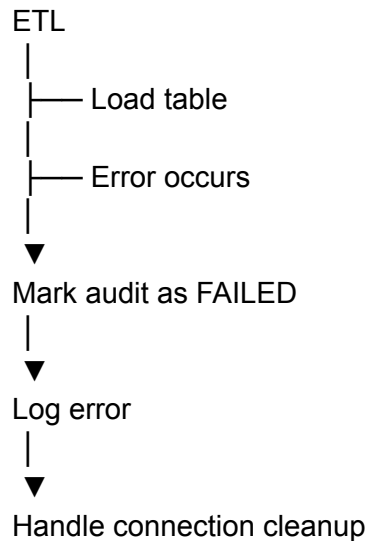
...

finally:

...

## Bronze Layer — Data Warehouse Project

If a table load fails:



The error is also stored in:

`error_message`

inside the audit table.

---

## 16. Transaction / Commit Strategy

The current implementation commits after each successful table:

```
load_table(...)
conn.commit()
```

Therefore each table load is treated independently.

For example:

```
crm_cust_info → SUCCESS → COMMIT
crm_prd_info → SUCCESS → COMMIT
crm_sales_details → SUCCESS → COMMIT
```

This is suitable for the current Bronze full-load implementation.

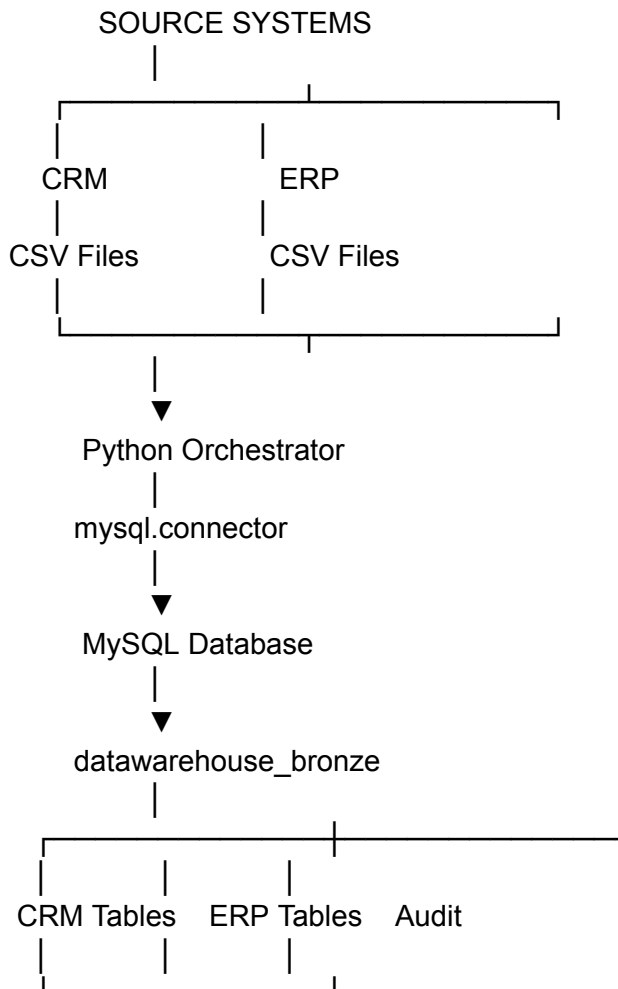
One important MySQL detail is that:

`TRUNCATE TABLE`

causes an implicit commit, so a rollback cannot restore the previous contents of a table after it has been truncated.

# 17. Bronze Pipeline Execution

The complete Bronze pipeline is:



# 18. Bronze Tables Successfully Loaded

The pipeline was successfully executed.

The final record counts were:

| Bronze Table               | Records Loaded |
|----------------------------|----------------|
| <code>crm_cust_info</code> | 18,494         |

## Bronze Layer — Data Warehouse Project

|                   |        |
|-------------------|--------|
| crm_prd_info      | 397    |
| crm_sales_details | 60,398 |
| erp_loc_a101      | 18,484 |
| erp_cust_az12     | 18,484 |
| erp_px_cat_g1v2   | 37     |

### Total Records

```
18,494
+ 397
+ 60,398
+ 18,484
+ 18,484
+ 37

116,294 records
```

So the current Bronze pipeline successfully loaded **116,294 records across six Bronze tables**.

---

## 19. Validation

After loading the data, record counts were checked using SQL.

Example:

```
SELECT 'crm_cust_info' AS table_name,
 COUNT(*) AS record_count
FROM datawarehouse_bronze.crm_cust_info
```

UNION ALL

```
SELECT 'crm_prd_info',
 COUNT(*)
FROM datawarehouse_bronze.crm_prd_info;
```

The same validation was performed for all six Bronze tables.

This confirms that the ingestion process successfully populated the Bronze tables.

## Bronze Layer — Data Warehouse Project

---